

Received 6 January 2026, accepted 17 January 2026, date of publication 4 February 2026, date of current version 9 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3659209

RESEARCH ARTICLE

Memory-Efficient Multi-Island Jaya Feature Selection for Large-Scale DDoS Attack Detection

GEETA SINGH¹ AND NEELU KHARE²

¹School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India

²School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India

Corresponding author: Neelu Khare (neelu.khare@vit.ac.in)

This work was supported by Vellore Institute of Technology, Vellore, Tamilnadu, India.

ABSTRACT Distributed Denial-of-Service (DDoS) attacks are among the most destructive events in the cyber realm, causing significant disruptions to the operations of organizations, communities, and individuals. These severe incidents occur when an attacker exhausts all resources of the targeted system, preventing legitimate users from accessing it. The primary challenges in recognizing these attacks are the memory constraints and scalability issues of current attack detection systems. To detect such large-scale attacks, a thorough analysis of multidimensional volumetric network traffic is conducted. However, the presence of substandard features within this traffic diminishes the effectiveness of detection. This paper introduces a feature selection pipeline that combines a memory-efficient D-optimal subsampling method with the Jaya optimization algorithm, designed to extract important features from the CIC-DDoS2019 dataset. The attack detection model employs this feature selection to create an automated, scalable classifier that uses the H2O Automated Machine Learning (AutoML) platform to identify large-scale DDoS attacks. This classifier effectively differentiates between eight distinct types of attacks and one benign traffic type. The model reports remarkable metrics, achieving a precision of 89.9%, a recall of 90.0%, an F1 score of 89.9%, and an accuracy of 89.9% in the cross-validation data. Furthermore, it surpasses the abilities of contemporary models.

INDEX TERMS DDoS attack, jaya algorithm, D-optimal subsampling, automated machine learning, CIC-DDoS2019 dataset, cyber security.

I. INTRODUCTION

The Covid-19 outbreaks were so compelling that people started working from home using various digital platforms, smart home devices, wearables, and smartphones. The necessity of a digitally connected world was the cause behind the adoption of digital transformation strategies. This emergency enormously increased web traffic. This sudden change imposed many challenges that hindered the smooth operation of the organizational / industrial data management and analysis process [1], [2]. This phenomenon is still a challenge and will continue to be a challenge in the future due to evolving technologies, human needs, the emergence of novel networking architectures, and many other factors.

The associate editor coordinating the review of this manuscript and approving it for publication was Ayman El-Baz³.

Consequently, digitally connected enterprises are seeking a protective umbrella against increasing exposure to network attacks [3], [4], [5], [6], [7], [8], [9]. The most prevalent and defamed DDoS attacks always make eye-catching headlines in the news. It works by transferring a large amount of network traffic towards the victim to exploit its resources and render it inaccessible to other authentic users. A delay in the detection of such attacks may cause the victim system to crash, resulting in significant monetary damages, downtime, and information loss, etc. The impact of DDoS attacks has grown exponentially. This trend continues to accelerate at an alarming rate. Finding a real-world solution for attack detection in the present scenario requires a model trained on a dataset that includes all the latest attack patterns with discriminative features for improved classification accuracy. The Canadian Institute for Cyber Security (CIC)

generated a dataset, CIC-DDoS2019, to address all the weaknesses in existing attack datasets and included numerous records of recent DDoS attack traces [10]. However, the higher dimensionality of this network traffic dataset imposes the need for proper data pre-processing and feature selection techniques. Traditional signature-based detection demonstrates high accuracy for recognized threats; however, it encounters difficulties with zero-day attacks, requires regular updates, and experiences performance degradation as its database expands. Machine learning (ML) enhances network intrusion detection (NID) by recognizing established and novel threats without the need for continuous updates, thus adapting to the changing landscape of cyber threats. However, conventional ML techniques depend on manual interventions, which complicate their implementation, reduce efficiency when handling large datasets, and can lead to decreased accuracy. In addition, the memory efficiency and scalability of detection systems are critical factors in the detection of large-scale DDoS attacks.

Taking into account all these challenging factors, we have developed an effective classifier model that provides a memory-efficient, scalable, and automated solution to detect DDoS attacks. Jaya is a metaheuristic algorithm renowned for its simplicity and effectiveness in extracting significant features [11], [12]. In this study, we propose a modified version of Jaya for the purpose of feature selection. Masking techniques in feature selection, particularly in relation to Jaya algorithms, refer to the methods used to convert continuous solution vectors (obtained from optimization algorithms like Jaya) into binary feature masks, indicating whether a feature is selected (1) or not (0). We utilized a Memory-Efficient D-optimal Mask Generator that employs D-optimal Subsampling for initial feature masking [13], [14], [15], [16]. This approach ensures the generation of a diverse yet statistically optimal initial population (set of masks) for feature selection. The application of this initial population improves the chances of better solution generation using the Jaya algorithm. Automated Machine Learning (AutoML) has emerged as a viable solution to address algorithm selection, hyperparameter optimization, feature engineering, scalability, and interpretability issues in machine learning tasks [17]. H2O AutoML, developed by H2O.ai, is a comprehensive AutoML platform that enables automated model training and hyperparameter optimization within a user-defined time frame. It features an intuitive interface and supports a wide range of algorithms, including deep learning models. H2O provides various model explainability techniques applicable to AutoML objects (collections of models) and individual models (such as the leader model). These explanations can be generated automatically with a single function call, offering an easy way to explore and clarify the AutoML models. With exceptional scalability, H2O AutoML can efficiently handle large datasets and complex models. In addition, it integrates advanced feature engineering methods and ensemble techniques to enhance model performance. Hence, with its exceptional features, the

AutoML framework is preferred for efficiently automating the detection of large-scale DDoS attacks.

This paper evaluates AutoML frameworks for DDoS attack detection in the CIC-DDoS2019 dataset with the best features generated by the Multi-Island Jaya pipeline, focusing on the creation of automated algorithms designed to improve performance. The significant contributions of the proposed work are outlined below:

- A memory-efficient and scalable feature selection method is introduced to create an AutoML optimized DDoS attack detection model.
- The dataset undergoes preprocessing and balancing before its use in evaluating the performance of the model.
- A pipelining strategy named ‘Multi-Island Jaya’ is applied to combine D-optimal Sub-sampling for initial population generation and Jaya optimization for the best feature selection using the initial population.
- A hybrid of the Multi-Island Jaya and H2O AutoML platform is used to identify the optimal classifier model trained on the feature-reduced dataset.
- The performance of the model is compared to that of other similar models.

The developed model effectively classifies eight types of attacks: “DrDoS_DNS”, “DrDoS_LDAP”, “DrDoS_MSSQL”, “DrDoS_NTP”, “DrDoS_NetBIOS”, “DrDoS_SSDP”, “DrDoS_UDP”, and “Syn”, as well as one benign network flow from the CIC-DDoS2019 dataset.

This article is structured into five sections, starting with Introduction Section I, which provides a brief overview of DDoS attacks, recent trends in DDoS detection, challenges faced, the motivation for this research, and our contributions. Section II reviews similar research articles related to DDoS classification. Section III offers a concise description of the methodologies used in the proposed work. The experimental setup and result comparisons are discussed in Section IV. Finally, Section V concludes the article by summarizing the findings of the proposed work and suggesting future directions in the relevant research field.

II. RELATED WORK

The CIC-DDoS2019 dataset offers valuable insights into the trends of recent DDoS attacks and can assist in the training and development of detection models aimed at addressing the severe challenges posed by large-scale DDoS attacks. Attack Detection Systems that utilize deep learning have become a compelling solution, thanks to their ability to identify intricate patterns within network traffic [18], [19]. However, the presence of high-dimensional input data, often characterized by redundant or irrelevant features, can impede detection efficiency and escalate computational costs, especially in resource-limited IoT gateways [20], [21], [22]. In the realm of big data, traditional framework-based classification methods have fallen prey to large-scale DDoS attacks. The traditional approach presents multiple challenges:

(1) It demands additional time for traffic analysis, (2) An explicit communication protocol must be established, (3) It is unable to analyze network traffic in real-time, among others.

A. H2O FRAMEWORK BASED ATTACK DETECTION

H2O is an open source library that provides essential services [23], [24], [25]. It enables the design of distributed models, offers high scalability features for the generated models, and provides support for generating highly scalable POJO (Plain Old Java Object) code. The H2O framework provides a platform for the development of distributed, scalable attack classifier models.

The message queue telemetry transport protocol (MQTT) is a widely accepted messaging protocol used to communicate data between devices. MQTT is susceptible to many types of attacks, including DDoS, Man-in-the-middle (MITM), brute-force authentication, MQTT publish flood attacks, etc. The H2O library was exploited to develop distributed machine learning classifier-based models, namely DRF, H2O Generalized Linear Models (H2OGLM), H2O Gradient Boosting Machine (H2OGBM), H2O XGBoost, and H2O Deep Learning (DL), which were capable of detecting attacks in a distributed environment in real time to protect the MQTT protocol in different networks [26]. Multiple metrics were used to evaluate the performance of the H2O models in the MQTTset dataset [27]. H2OXGBoost demonstrated its proficiency in MQTT protocol attack classification. H2OXGBoost scored MSE 0.101, RMSE 0.318, Log Loss 0.371, and Mean per class error 0.255.

Patil et al. [28] proposed KS-DDoS, a distributed classification approach based on the Apache Kafka stream for real-time DDoS attack classification using three highly scalable H2O machine learning algorithms, namely DRF, gradient boosting machine (GBM), and NB. Hadoop clusters were utilized to train H2O ML algorithms on Kafka stream clusters. In a self-operating manner, CICFlowMeter-V4.0 immediately ingested incoming network traces; the produced network flows were published on Kafka topics. In a self-operating manner, CICFlowMeter-V4.0 immediately ingested incoming network traces; the produced network flows were published on Kafka topics. The distributed classifier models were fed with extracted significant normalized features for analysis. Later, these classifiers published the classification results on the Kafka topic. All of these operations were performed in a distributed manner across multiple Kafka Stream cluster nodes. KS-DDoS was designed and validated using the CIC-DDoS2019 dataset. Nine types of network traces, namely Benign, DDoS_DNS, DDoS_LDAP, DDoS_MSSQL, DDoS_NetBIOS, DDoS_UDP, DDoS_NTP, DDoS_SSDP, and Syn, were considered for the development and deployment of the classification model. DRF outperforms the other H2O ML algorithms. The average classification accuracy reached 80.07% for DRF, followed by GBM-79.94% and NB-59.55%.

SSK-DDoS is another distributed real-time classification approach in which preprocessing and classification are distributed among Spark cluster nodes. In a self-operating manner, incoming network flows are presented on Kafka topics, essential variables are selected to formulate features, and classification and predictions are performed and published on the Kafka topics for further analysis. The Spark MLlib library includes a variety of distributed machine learning algorithms. SSK-DDoS utilized multinomial logistic regression (MLR), DecisionTreeClassifier (DTC), RF, and NB algorithms. RF performed better in all scenarios. These models were validated using the CIC-DDoS2019 dataset. Seven types of network traces, namely Benign, DDoS_DNS, DDoS_LDAP, DDoS_MSSQL, DDoS_NetBIOS, DDoS_UDP, and Syn, were considered for the development and deployment of a classification model [29].

B. AutoML FRAMEWORK BASED ATTACK DETECTION

AutoML techniques present promising solutions for developing autonomous attack Detection Systems (IDSs) by automating the laborious processes involved in the data analytics and machine learning pipeline. AutoML; a relatively new field of research, has already been utilized in numerous recent studies to develop autonomous attack detection models tailored for contemporary networks. Singh et al. [30] introduced AutoML-ID, an IDS based on AutoML specifically designed for Wireless Sensor Networks (WSNs). The AutoML-ID methodology emphasizes straightforward automated machine learning model selection and hyperparameter optimization through Bayesian Optimization (BO). The AutoML-ID model was evaluated using a public IDS dataset, Intrusion-Data-WSN, and demonstrated superior performance compared to traditional machine learning models. Elmasry et al. [31] proposed a dual Particle Swarm Optimization (PSO) and Deep Learning-based IDS for network intrusion detection. This approach employs the PSO method to select features and fine-tune hyperparameters for three deep learning techniques: Deep Neural Networks (DNN), Long Short-Term Memory (LSTM), and Deep Belief Networks (DBN). This IDS model surpasses other methods in terms of accuracy and detection rate when tested on the CIC-IDS2017 dataset. Khan et al. [32] introduced an Optimized Ensemble IDS (OE-IDS) aimed at intrusion detection within network environments. OE-IDS automates the hyperparameter tuning for four supervised machine learning algorithms and utilizes them to construct an ensemble model based on a soft-voting strategy. The OE-IDS model achieved higher accuracy and f1 scores than most other traditional machine learning models when evaluated on the CICIDS2017 and UNSW-NB15 datasets.

C. RESEARCH GAP

The current body of research has shown the benefits of using the H2O platform and AutoML for attack detection, particularly in improving performance and minimizing

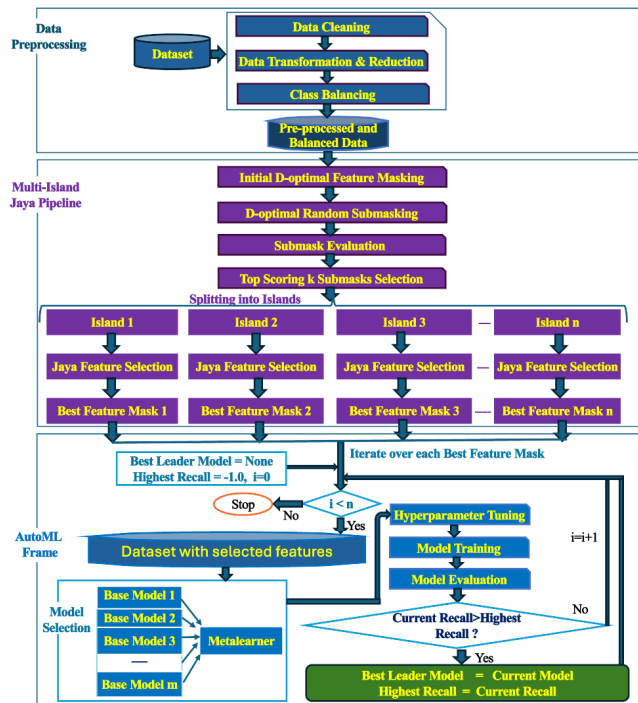


FIGURE 1. System workflow.

the need for human intervention in attack detection and cyber security. However, numerous existing AutoML-based IDS models primarily concentrate on automating model selection and hyperparameter tuning, leaving considerable room for enhancement in other vital phases of the AutoML process. In our suggested AutoML framework, we introduce a memory-efficient and scalable feature selection technique to increase the overall performance of the model.

III. METHODOLOGY

In this section, we present a feature selection method aimed at identifying significant features from the CIC-DDoS2019 dataset to create a classification model for detecting benign traffic and eight types of attacks: DDoS_DNS, DDoS_LDAP, DDoS_MSSQL, DDoS_NTP, DDoS_NetBIOS, DDoS_SSDP, DDoS_UDP, and Syn. We utilized the H2O AutoML platform for the implementation of DDoS attack classifier models and the final classifier evaluation. The approach is tailored for large-scale datasets such as CIC-DDoS2019. The system workflow is given in figure 1.

A. DATASET PREPARATION

The CIC-DDoS2019 dataset comprises normal flow and modern DDoS attack flows, which resemble real novel network traffic. New attacks and related taxonomies are defined with reference to reflection-based and exploitation-based attacks. Related documentation is available at [10]. This dataset was created by conducting 7 DDoS attacks

on the first day and 12 DDoS attacks on the second day. In this study, second-day attacks are considered, including benign traffic and only 8 specific attacks: DDoS_DNS, DDoS_LDAP, DDoS_MSSQL, DDoS_NTP, DDoS_NetBIOS, DDoS_SSDP, DDoS_UDP, and Syn from a total of 12 attacks.

1) PREVENTING DATA LEAKAGE

Data leakage occurs when information from the test set is unintentionally accessible to the learning algorithm during the training phase, which can lead to artificial improvement of performance metrics and undermine generalization. To maintain the validity and reproducibility of the experimental findings, special attention was paid to preventing any potential data leakage between the training and testing phases.

- 1) The dataset was partitioned into training sets (70%) and testing sets (30%) utilizing a stratified sampling approach to maintain the original class distribution across the divisions.
- 2) Feature preprocessing procedures, including normalization, scaling, and feature selection, were conducted solely on the training set. Transformation parameters (in this case, standard deviation for scaling) were calculated exclusively on the training data and subsequently applied to the testing data. This ensures that no statistical information from the test set affected the training process.
- 3) The features that posed a significant risk of leakage were omitted from the analysis. These features include unique identifiers (e.g., 'FlowID'), source and destination IP addresses ('SourceIP', 'DestinationIP'), timestamps ('Timestamp'), and vague or non-informative attributes (e.g., Similar-HTTP, Unnamed:0). The removal of such fields guarantees that the classifier learns patterns of traffic behavior instead of memorizing identifiers.
- 4) Lastly, constant or nearly constant features and those with exceedingly high cardinality were eliminated by variance thresholding and exploratory analysis. This process reduces noise while further decreasing the risk of leakage from features that trivially correlate with attack labels.

Through a thorough analysis of the dataset and following the steps outlined below, the dataset has been structured for the purpose of creating and evaluating the XGBoost-based ensemble machine learning attack classifier.

- 1) 'FlowID' functions solely as an identifier. The features 'FwdAvgBytes/Bulk', 'FINFlagCount', 'FwdAvgBulkRate', and 'PSHFlagCount' demonstrate low variance and are mostly zero. The 'Timestamp' values are monotonically increasing, which may result in leakage or bias. 'SourceIP' and 'DestinationIP' are considered unreliable, as attackers have the

TABLE 1. Dataset class label mapping.

Actual Class Labels	Encoded Labels	Actual Class Labels	Encoded Labels
BENIGN	0	DrDoS_NetBIOS	5
DrDoS_DNS	1	DrDoS_SSDP	6
DrDoS_LDAP	2	DrDoS_UDP	7
DrDoS_MSSQL	3	Syn	8
DrDoS_NTP	4		

capability to spoof IP addresses. ‘Simillar-HTTP’ is a synthetic feature derived from application-layer heuristics (number of similar HTTP requests in a time window). ‘Simillar-HTTP’ has unclear semantics within the dataset, and not all flows are HTTP-based. Furthermore, CIC does not fully document how SimillarHTTP was generated (without open-source code or reproducible methodology). This opacity limits the interpretability of the model and the reproducibility of the dataset. ‘Unnamed:0’, ‘FlowID’, ‘SourceIP’, ‘DestinationIP’, ‘FwdAvg-Bytes/Bulk’, ‘FINFlagCount’, ‘FwdAvgBulkRate’, ‘PSHFlagCount’, ‘Timestamp’, and ‘Simillar-HTTP’ (10 features), which include IDs, flow identifiers, and features with high cardinality or constant values, are eliminated.

- 2) The ‘Label’ feature, which is our focus, is categorical and requires encoding. This feature is encoded using the LabelEncoder from the scikit-learn (referred to as sklearn) Python library (see Table 1).
- 3) The StandardScaler method from the same Python library is employed to standardize features according to the equation. 1.

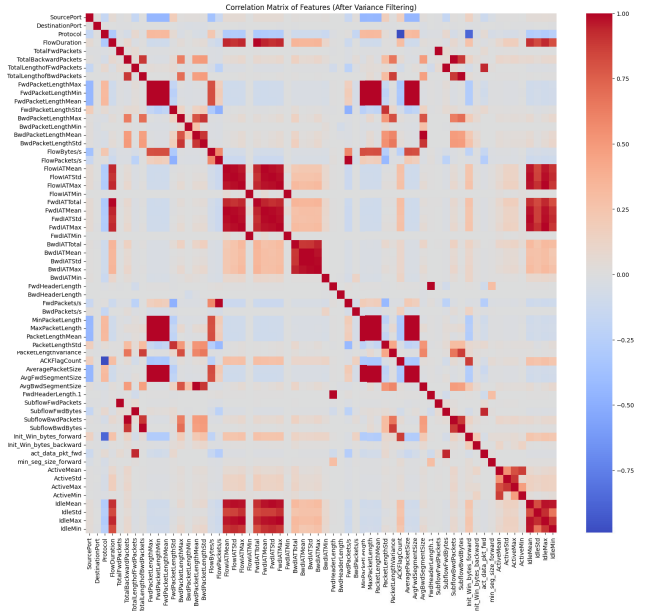
$$z = \frac{x - u}{s} \quad (1)$$

Here:

- u - Mean of the training examples
- s - Standard deviation of the training examples.

- 4) We apply the corr() method from the Pandas DataFrame to calculate the absolute correlation matrix, allowing for the analysis of pairwise correlations among the features of the dataset. A total of 30 features with corr() values greater than 0.95 have been identified (see figure 2) and subsequently removed due to their high correlation.
- 5) The dataset, now comprising 47 features and one target, is downsampled using the RandomUnderSampler method from the imbalanced-learn Python package to ensure 28,849 records for each attack flow and one benign flow.

As a result, this preprocessed dataset, which includes 47 features and one target feature ‘Label’, comprises a total of 259,641 samples. The table 2 illustrates these 47 features. This preprocessed dataset is utilized in the development of the attack classifier.

**FIGURE 2.** CIC-DDoS2019 correlation matrix after variance filtering.**TABLE 2.** 47 features of preprocessed CIC-DDoS 2019 dataset.

Indices	Features	Indices	Features
1	ActiveMax	25	FlowIATMean
2	ActiveMean	26	FlowIATMin
3	ActiveMin	27	FlowPackets/s
4	ActiveStd	28	FwdAvgPackets/Bulk
5	BwdAvgBulkRate	29	FwdHeaderLength
6	BwdAvgBytes/Bulk	30	FwdPacketLengthMax
7	BwdAvgPackets/Bulk	31	FwdPacketLengthStd
8	BwdHeaderLength	32	FwdPSHFlags
9	BwdIATMean	33	FwdURGFlags
10	BwdIATMin	34	IdleStd
11	BwdIATTotal	35	Inbound
12	BwdPacketLengthMax	36	Init_Win_bytes_backward
13	BwdPacketLengthMean	37	Init_Win_bytes_forward
14	BwdPacketLengthMin	38	min_seg_size_forward
15	BwdPacketLengthStd	39	PacketLengthStd
16	BwdPackets/s	40	PacketLengthVariance
17	BwdPSHFlags	41	Protocol
18	BwdURGFlags	42	SYNFlagCount
19	CWEFlagCount	43	TotalBackwardPackets
20	DestinationPort	44	TotalFwdPackets
21	Down/UpRatio	45	TotalLengthofFwdPackets
22	ECEFlagCount	46	URGFlagCount
23	FlowBytes/s	47	SourcePort
24	FlowDuration		

B. MULTI-ISLAND JAYA PIPELINE

The proposed work presents a memory-efficient pipeline for feature selection, combining randomized D-optimal subsampling and the Multi-Island Jaya optimization.

Pipeline Structure:

- 1) Generation of initial D-optimal subsamples: Apply the randomized D-optimal subsampling strategy to generate the top k initial diverse binary masks.
- 2) Islands Creation: Distribute the top k binary masks into n islands (subpopulations).

- 3) Best feature mask generation with the Jaya Algorithm:
Apply the Jaya algorithm on each island to produce the best mask from all islands.

1) GENERATION OF INITIAL D-OPTIMAL SUBSAMPLES

In order to create high-quality and memory-efficient initial feature masks, we employed a randomized D-optimal subsampling algorithm.

Intuition Behind D-Optimality

- To prevent exponential explosion from full factorial designs, a randomized approach is used to generate binary feature masks
- For each mask, i create a subset $\mathbf{F}_i$ of features $\mathbf{F}$
- Select the feature subsets that maximize the volume of the confidence ellipsoid formed by the features; that is, the largest spread in the feature space.
- The determinant of $\mathbf{F}_i^T \mathbf{F}_i$ is proportional to the volume of the confidence ellipsoid.
- A higher determinant indicates a more diverse or informative feature subset, which is better for modeling.

The following are the steps to calculate the determinant of $\mathbf{F}_i^T \mathbf{F}_i$:

- 1) Compute the information matrix using equation (2):

$$\mathbf{A}_i \leftarrow \mathbf{F}_i^T \mathbf{F}_i + \epsilon \mathbf{I} \quad (2)$$

- 2) Set the score using equation (3):

$$\mathbf{S}_i \leftarrow \det(\mathbf{A}_i) \quad (3)$$

Here:

- $\mathbf{A}_i$: information matrix
- $\mathbf{F}_i^T$: Transpose of $\mathbf{F}_i$
- $\mathbf{F}_i^T \mathbf{F}_i$: The feature covariance matrix is computed for each selected mask to evaluate D-optimality via the determinant.
- $\mathbf{I}$: Identity matrix
- ϵ : A small positive constant (e.g. 1×10^{-6})
- $\mathbf{S}_i$: The determinant of the information matrix $\mathbf{A}_i$ is used as a criterion for D-optimality.

The randomized generation step enables efficient exploration of the combinatorial mask space, while determinant-based scoring promotes the selection of linearly independent and information-rich feature subsets. Adding $\epsilon \mathbf{I}$ inflates diagonal elements just enough to break exact dependencies without significantly altering the structure of $\mathbf{F}_i^T \mathbf{F}_i$. The objective is to choose k binary masks (feature subsets) from an extensive pool of candidates that maximizes the determinant of the information matrix, thus enhancing diversity and information richness. The steps of the memory-efficient D-optimal subsampling method are outlined in Algorithm 1.

2) ISLAND GENERATION

The generated top k masks are divided into multiple subpopulations (n islands), each consisting of k/n feature masks. The Jaya optimization algorithm is applied independently on each island.

Algorithm 1 Memory-Efficient D-Optimal Subsampling

Require: Number of features $\mathbf{F}$, total samples N , top k masks, minimum features m , random seed s

Ensure: k binary masks with highest D-optimal scores

- 1: Initialize random seed s
- 2: Generate N binary masks $\mathbf{M} \in \{0, 1\}^{N \times F}$
- 3: **for** $i = 1$ to N **do**
- 4: Extract mask $\mathbf{m}_i \leftarrow \mathbf{M}[i]$
- 5: **if** $\sum \mathbf{m}_i < m$ **then**
- 6: Set score $\mathbf{S}_i \leftarrow 0$
- 7: **else**
- 8: Select subset $\mathbf{F}_i \leftarrow \mathbf{M}[:, m_i]$
- 9: Compute information matrix using equation 2
- 10: Set score using equation 3
- 11: **end if**
- 12: **end for**
- 13: Select indices of top k scores from $\{\mathbf{S}_i\}_{i=1}^N$
- 14: **return** corresponding top scoring k binary masks

3) BEST FEATURE MASK GENERATION WITH JAYA ALGORITHM

The Jaya algorithm is a population-oriented optimization method that progressively refines candidate solutions through the update equation 4, with the aim of converging towards the best known solution while steering clear of the least favorable solution. Below are the steps to execute the Jaya Optimization Algorithm for the purpose of identifying optimized hyperparameters.

- Train the classification model using each mask from the island. Assess and record the performance score (e.g., Random Forest Classifier's recall_rate) of the feature mask.
- Determine the best and worst masks within the island.
- Update each candidate solution employing the Jaya update equation (4):

$$\mathbf{P}_{nw} = \mathbf{P}_{cr} + r_1 \cdot (\mathbf{P}_{bs} - |\mathbf{P}_{cr}|) - r_2 \cdot (\mathbf{P}_{wr} - |\mathbf{P}_{cr}|) \quad (4)$$

here:

- $\mathbf{P}_{cr}$ is the current population.
- $\mathbf{P}_{bs}$ are the best solutions in the current population
- $\mathbf{P}_{wr}$ are the worst solutions in the current population
- r_1 and r_2 are random numbers in $[0, 1]$.
- Apply the sigmoid function and thresholding to convert each component f_i of continuously updated solutions in the population into binary masks using equations 5 and 6

$$\sigma(f_i) = \frac{1}{1 + e^{-f_i}} \quad (5)$$

$$f_i^{\text{binary}} = \begin{cases} 1, & \text{if } \sigma(f_i) > \text{rand}(0, 1) \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

- Develop new models utilizing revised masks. Assess performance once more.
- Proceed with iterations until a predetermined maximum number of generations is attained.

The steps of the multi-Island Jaya pipeline algorithm are outlined in Algorithm 2.

Algorithm 2 Multi-Island Jaya Pipeline With Randomized D-Optimal Initialization

Require: Dataset $\mathcal{X} \in \mathbb{R}^{n \times d}$, labels y , number of islands I , D-optimal samples S , top scoring k binary masks, generations G

Ensure: List of best binary masks from each island

```

1: Generate  $S$  random binary masks  $\mathbf{M} \in \{0, 1\}^{S \times d}$  with  $d$ 
   dimension
2: for each mask  $\mathbf{m}_i \in \mathbf{M}$ ,  $i = 1$  to  $S$  do
3:   if number of selected features in  $\mathbf{m}_i < \text{threshold}$  then
4:     Set score  $s_i \leftarrow 0$ 
5:   else
6:     Extract subset  $\mathbf{M}_{\text{sub}} = \mathbf{M}[:, m_{ij} = 1], j = 1$  to  $d$ 
7:     Compute information matrix:  $\mathbf{A}_i \leftarrow \mathbf{M}_{\text{sub}}^T \mathbf{M}_{\text{sub}} + \epsilon \mathbf{I}$ 
8:     set score  $s_i \leftarrow \det(\mathbf{A}_i)$ 
9:   end if
10: end for
11: Select top  $k$  masks with highest  $s_i$  to form  $\mathbf{M}_{\text{top}}$ 
12: Split  $\mathbf{M}_{\text{top}}$  into  $I$  islands
13: for each island  $i = 1$  to  $I$  do
14:   Initialize population  $\mathbf{P}_i$  from island  $i$ 's masks
15:   for generation  $g = 1$  to  $G$  do
16:     Evaluate fitness of all individuals in  $\mathbf{P}_i$ 
17:     Identify best and worst individuals
18:     for each individual  $\mathbf{p}_j$  in  $\mathbf{P}_i$  do
19:       Update  $\mathbf{p}_j$  using Jaya rule with random numbers
          $r_1, r_2 \leftarrow \text{random}(0, 1)$ :
20:        $\mathbf{p}_j \leftarrow \mathbf{p}_j + r_1 \cdot (\text{best} - |\mathbf{p}_j|) - r_2 \cdot (\text{worst} - |\mathbf{p}_j|)$ 
21:       Apply sigmoid:  $\mathbf{p}_j \leftarrow \sigma(\mathbf{p}_j)$ 
22:       Binarize:  $\mathbf{p}_j \leftarrow (\mathbf{p}_j > \text{rand}(0, 1))$ 
23:       Mutate one random bit with small probability
24:     end for
25:   end for
26:   Store best solution from  $\mathbf{P}_i$  as  $b_i$ 
27: end for
28: return  $\{b_1, b_2, \dots, b_I\}$ 

```

C. H2O AutoML PLATFORM

The best AutoML model is constructed using the best identified feature masks. H2O AutoML can facilitate the automation of the machine learning workflow, including the automatic training and tuning of various ML models within a user-defined time frame. H2O AutoML features Random Forests (both Default and Extremely Randomized Tree types), XGBoost Gradient Boosting Machines, H2O Gradient Boosting Machines, Deep Neural Networks, and Generalized Linear Models as its foundational models. H2O AutoML trains and cross-validates these models using default configurations. Following the training of the base models, stacked ensemble models are developed using the H2O

Stacked Ensemble algorithm, which involves training a second-level metalearner to determine the best combination of the base learners. A random search across various algorithm families yields a highly diverse array of base models, and when combined with stacking, results in robust ensembles. This stacking approach has been demonstrated to represent an asymptotically optimal learning system. AutoML machine learning models are trained on each of the best feature masks to obtain the final improved best AutoML leader model. The model with the best recall score is selected as the best AutoML leader model. The performance of the best AutoML leader model is evaluated and discussed in the next section.

IV. EXPERIMENTAL SETUP AND RESULT DISCUSSION

All experiments were conducted using a Google Colab notebook with Google Cloud Storage (GCS) support. The CICDDoS2019 dataset is preprocessed, scaled, and balanced before being uploaded to the Google Cloud bucket. The uploaded data are accessed and downloaded in the Colab session for model development and evaluation. The multi-Island Jaya pipeline algorithm receives pre-processed balanced data as input. D-optimal subsampling is the first step to be executed in the Jaya pipeline. It evaluates the initial binary masks produced randomly, selects the top k binary masks, and divides them into n islands with k/n masks in each. The n number of Jaya algorithms utilizes these islands as initial feature masks to select features from input data and later assess their fitness score in parallel, applying updates until a stopping criterion (maximum number of generations) is satisfied. The n islands generate the best n feature-sets. AutoML algorithms are trained and evaluated iteratively on n datasets with n best feature sets to ultimately produce the best AutoML leader model with the highest recall rate.

A. PROPOSED METHODOLOGICAL FRAMEWORK

1) D-OPTIMAL SUBSAMPLING FOR GENERATION OF OPTIMUM POOL OF FEATURESETS

Purpose: Reduce redundancy and improve the representativeness of samples for high-dimensional datasets.

- Theory: D-optimal design selects a subset of samples that maximizes the determinant of the information matrix $\mathbf{F}_i^T \mathbf{F}_i$, thus preserving the geometric structure of the data.
- D-optimal Approach:
 - Calculate the determinant of the information matrix $\mathbf{F}_i^T \mathbf{F}_i$ for each binary sample $\mathbf{F}_i$, $i \in [1, 5000]$
 - choose 20 binary samples that maximize the determinant of the information matrix.
- Objective:
 - Selection of 20 binary masks (feature subsets) from 5000 candidates to enhance diversity and information richness.
- Protocol:

- Use the D-optimal criterion to select 20 optimal subsets from 5000 randomly generated binary samples of dimension 47.
- Use these optimal subsets as the initial population for the proposed MIJ.
- Tools: Python implementation.

2) MULTI-ISLAND JAYA ALGORITHM FOR FEATURE SELECTION

Purpose: Avoid local optima and explore the solution space effectively to select informative features.

- Base Algorithm: Jaya algorithm (no parameter tuning; minimizes/maximizes the objective by moving toward the best and away from the worst solution).
- Multi-Island Approach:
 - Use 20 optimal feature sets of the dataset produced by the D-optimal criterion as the MIJ population.
 - Split the population into multiple islands (sub-populations).
 - Apply Jaya locally in each.
 - Use these populations (feature sets) to create 20 subsets (20/4 = 5 subsets for each island) of the dataset.
 - Periodically migrate the best candidates across islands.
- Objective Function:
 - Maximize classification accuracy on the validation set.

Protocol:

- Initialize the population with binary vectors (feature masks).
- Each island evolves in parallel (epochs).
- Perform migration every M generations.
- Evaluate using a random forest classifier.

3) INTEGRATION INTO H2O AutoML

Purpose: Evaluate the selected feature subset using robust automated ensemble learning pipelines.

- Approach:
 - Use H2O AutoML to:
 - * Automatically train multiple models: Distributed Random Forest (DRF), Gradient-Boosting-Machine (GBM), Generalized Linear Model (GLM), eXtremely Randomized Trees (XRT), XGBoost, Deep Learning (DL), and Stacked Ensembles.
 - * Rank the models according to their performance on the validation set.
 - Use the feature subset F_{opt} selected by the Jaya algorithm.
- Experimental Design:
 - Split the original dataset into Train, Validation, and Test.
 - Train Jaya on Train + Validation.

TABLE 3. 35 Best features selected by multi-island jaya algorithm.

S.No.	Features	S.No.	Features
1	SourcePort	19	FwdHeaderLength
2	DestinationPort	20	BwdHeaderLength
3	FlowDuration	21	BwdPackets/s
4	TotalFwdPackets	22	PacketLengthStd
5	TotalBackwardPackets	23	PacketLengthVariance
6	TotalLengthofFwdPackets	24	SYNFlagCount
7	FwdPacketLengthStd	25	URGFlagCount
8	BwdPacketLengthMax	26	CWEFlagCount
9	BwdPacketLengthMin	27	Down/UpRatio
10	BwdPacketLengthStd	28	BwdAvgBytes/Bulk
11	FlowBytes/s	29	BwdAvgPackets/Bulk
12	FlowIATMean	30	BwdAvgBulkRate
13	FlowIATMin	31	Init_Win_bytes_forward
14	BwdIATMean	32	min_seg_size_forward
15	FwdPSHFlags	33	ActiveMean
16	BwdPSHFlags	34	ActiveStd
17	FwdURGFlags	35	IdleStd
18	BwdURGFlags	-	-

- Use AutoML only on data filtered by F_{opt} .
- Evaluate the best H2O model on the test set.
- Performance Metrics:
 - Accuracy, Precision, Recall, F1-score
 - Time-to-convergence (3600 s)

The best AutoML Model is generated with 35 (out of 47) features. These best features are listed in Table 3.

B. RESULT ANALYSIS

Cross-validation is a statistical method employed in machine learning to assess a model's performance and determine its ability to generalize to new, unseen data. This technique is essential for identifying and addressing issues such as overfitting or underfitting. The fundamental concept involves repeatedly dividing the dataset into several subsets, known as "folds," for both training and testing purposes, and then averaging the outcomes to obtain a more reliable estimate of model performance compared to a single train-test split. Consequently, cross-validation helps minimize overfitting, offers a reliable performance estimate, utilizes data effectively, and supports model selection. H2O AutoML predominantly uses k -fold cross-validation as its standard method for evaluating model performance and ranking them on the leaderboard. By default, H2O AutoML performs 5-fold cross-validation. For each candidate model, H2O constructs $k + 1$ models: k individual models (each trained on $k - 1$ folds) and one main model trained on the complete training dataset. When cross-validation is activated, H2O employs cross-validation metrics for early stopping to avert overfitting. The predictions from cross-validation (holdout predictions) generated by the base models are automatically utilized to train the Stacked Ensemble models. The best AutoML leader model, intermediate, and final results are uploaded to GCS for future availability. The summary of the best AutoML leader model is shown in Table 4.

Figure 3 represents the confusion matrix in the cross-validation data [33].

TABLE 4. AotoML leader model summary.

Model Summary for Stacked Ensemble:	
key	value
Stacking strategy	cross_validation
Number of base models (used / total)	3/6
GBM base models (used / total)	1/1
XGBoost base models (used / total)	1/1
DRF base models (used / total)	0/2
DeepLearning base models (used / total)	1/1
GLM base models (used / total)	0/1
Metalearner algorithm	GLM
Metalearner fold assignment scheme	Random
Metalearner nfolds	5

	0	1	2	3	4	5	6	7	8
0	20297	1	0	0	3	2	5	1	0
1	1	19813	241	16	47	17	21	0	0
2	0	243	19833	11	7	0	85	3	0
3	0	7	519	19472	4	47	94	55	0
4	12	49	0	5	20069	4	1	9	1
5	3	32	0	1241	11	18895	11	9	0
6	2	59	592	223	6	1	11801	7416	1
7	0	2	19	177	10	3	6864	13074	1
8	0	1	0	2	3	1	1	1	20291

FIGURE 3. Confusion matrix on cross validation data.

Row labels are the actual class labels, and column labels are the predicted class labels. The performance of the model is evaluated using precision, recall, f1 score, and accuracy.

- Precision vs. Recall Clarified

- Precision (Quality):

- * It addresses how frequently the model is accurate”.
- * It focuses on reducing false positives (incorrectly identifying negatives as positives).

- Recall (Quantity/Sensitivity):

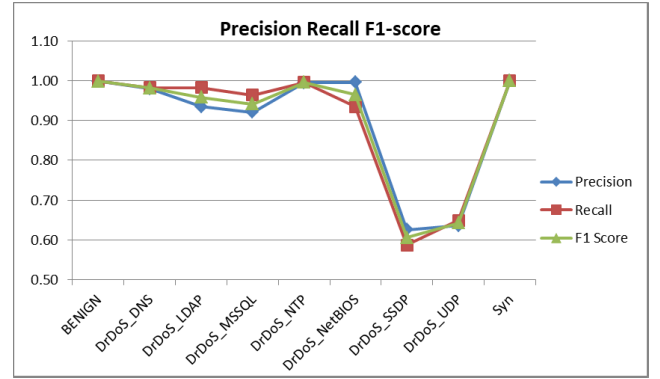
- * It addresses: “Out of all the genuine positive instances, how many did the model detect?”.
- * It focuses on reducing false negatives.

The f1 score is a vital machine learning metric that reconciles precision (the correctness of positive predictions) and recall (the ability to identify all positive instances) into a singular value, calculated as their harmonic mean. Accuracy refers to overall correctness and is particularly beneficial for balanced datasets. The f1 score is essential for assessing imbalanced datasets, where classes possess unequal samples, as it offers a more accurate measure than mere precision by taking into account both false positives and false negatives, with a scoring range from 0 (worst) to 1 (best). The equations for these metrics are given in (7), (8), (9) and (10).

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i} = \frac{\text{Diagonal}_i}{\text{Column}_i} \quad (7)$$

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i} = \frac{\text{Diagonal}_i}{\text{Row}_i} \quad (8)$$

$$\text{F1_Score}_i = 2 \cdot \frac{\text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \quad (9)$$

**FIGURE 4.** Class-wise precision, recall, and f1 score on the cross-validation data.**TABLE 5.** Class-wise precision, recall, and f1 score on the cross-validation data.

Class	Precision	Recall	F1 Score
BENIGN	1.00	1.00	1.00
DrDoS_DNS	0.98	0.98	0.98
DrDoS_LDAP	0.94	0.98	0.96
DrDoS_MSSQL	0.92	0.96	0.94
DrDoS_NTP	1.00	1.00	1.00
DrDoS_NetBIOS	1.00	0.94	0.96
DrDoS_SSDP	0.63	0.59	0.61
DrDoS_UDP	0.64	0.65	0.64
Syn	1.00	1.00	1.00
Average	0.90	0.90	0.90

$$\text{Accuracy} = \frac{\sum \text{Diagonal}}{\sum \text{Matrix}} \quad (10)$$

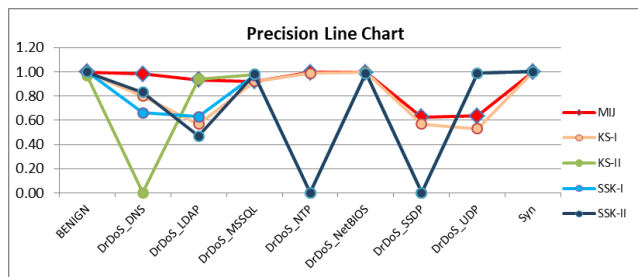
here:

- TP_i represents the true positives for the class i . It is the count of correctly predicted instances of class i .
- FP_i represents the false positives for the class i . It is the count of instances incorrectly predicted as class i .
- FN_i represents the false negatives for the class i . It is the count of incorrectly predicted instances in class i .
- *Accuracy* represents the ratio between the correct predictions and the total number of predictions (or the total records to be predicted in the exploited dataset).
- Diagonal_i represents the diagonal value i^{th} in the confusion matrix
- Column_i represents the total count of the values in column i^{th} of the confusion matrix.
- Row_i represents the total count of the values in row i^{th} of the confusion matrix.
- $\sum \text{Diagonal}$ represents the total count of all diagonal values in the confusion matrix.
- $\sum \text{Matrix}$ represents the total count of all values in the confusion matrix.

Figure 4 and Table 5 represent the precision, recall, and f1 score per class achieved by the proposed model on the cross-validation data. The model achieves impressive metrics, including 89.9% precision, 90.0% recall, 89.9% f1 score, and 89.9% accuracy, as reported on cross-validation data.

TABLE 6. Model performance comparison.

Precision									
Class	0	1	2	3	4	5	6	7	8
MIJ	1.00	0.98	0.94	0.92	1.00	1.00	0.63	0.64	1.00
KS-I	1.00	0.8	0.57	0.92	0.99	1.00	0.57	0.53	1.00
KS-II	0.97	-	0.94	0.98	-	1.00	-	0.99	1.00
SSK-I	1.00	0.66	0.63	0.97	-	0.99	-	0.99	1.00
SSK-II	1.00	0.83	0.47	0.98	-	0.99	-	0.99	1.00
Recall									
Class	0	1	2	3	4	5	6	7	8
MIJ	1.00	0.98	0.98	0.96	1.00	0.94	0.59	0.65	1.00
KS-I	1.00	0.3	0.93	0.96	1.00	0.99	0.17	0.86	1.00
KS-II	1.00	-	0.93	0.96	-	0.99	-	0.68	1.00
SSK-I	1.00	0.57	0.73	0.95	-	0.99	-	0.99	1.00
SSK-II	1.00	0.64	0.73	0.95	-	0.99	-	0.99	1.00
F1 score									
Class	0	1	2	3	4	5	6	7	8
MIJ	1.00	0.98	0.96	0.94	1.00	0.96	0.61	0.64	1.00
KS-I	1.00	0.44	0.71	0.94	1.00	0.99	0.26	0.64	1.00
KS-II	0.99	-	0.94	0.97	-	0.99	-	0.81	1.00
SSK-I	1.00	0.61	0.68	0.96	-	0.99	-	0.99	1.00
SSK-II	1.00	0.72	0.57	0.97	-	0.99	-	0.99	1.00

**FIGURE 5. Comparison of precision of proposed method with KS-DDoS and SSK-DDoS.**

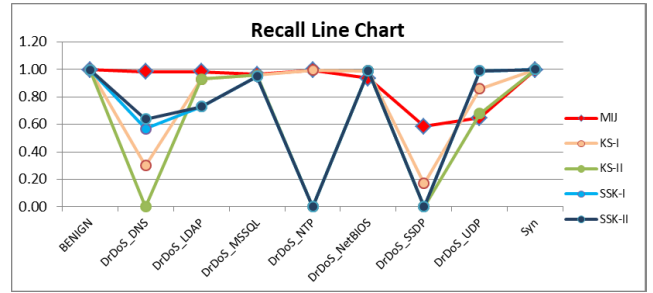
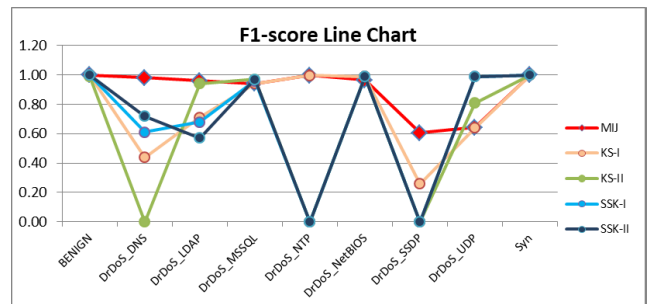
The proposed model demonstrates an average precision, recall, f1 score, and accuracy of 90%. It achieves per-class precision, recall, and f1 scores exceeding 0.92, 0.94, and 0.94 in six classes: DDoS_DNS, DDoS_LDAP, DDoS_MSSQL, DDoS_NTP, DDoS_NetBIOS, DDoS_SYN, along with one Benign class. The performance of this model is evaluated against relevant recent studies [28] and [29]. The results of this comparison are presented in Table 6.

Here:

- MIJ → Proposed Memory-Efficient Scalable Multi-Island Jaya based model
- KS-I → Performance evaluation of KS-DDoS [28] For Case-I, while designing
- KS-II → performance evaluation of KS-DDoS [28] for Case-II (After deployment)
- SSK-I → Performance evaluation of SSK-DDoS [29] For Case-I, while designing
- SSK-II → performance evaluation of SSK-DDoS [29] for Case-II (After deployment)

The proposed MIJ model exhibits enhanced performance compared to the KS-DDoS and SSK DDoS models. Figures 5, 6, and 7 clearly illustrate these comparisons.

When comparing MIJ with the KS-DDoS and SSK DDoS models, it achieves the following:

**FIGURE 6. Comparison of recall of proposed method with KS-DDoS and SSK-DDoS.****FIGURE 7. Comparison of f1 score of proposed method with KS-DDoS and SSK-DDoS.**

- the highest precision score, excluding DDoS_MSSQL and DrDoS_UDP
- the highest recall scores, with the exception of DrDoS_NetBIOS and DrDoS_UDP
- the highest f1 scores, except for DDoS_MSSQL, DrDoS_NetBIOS, and DrDoS_UDP

SSDP (Simple Service Discovery Protocol) is used for device discovery in UPnP (Universal Plug and Play) environments, mainly over UDP port 1900. UDP flood attacks are generic denial-of-service attacks that overwhelm a target with UDP packets, often across random ports. In the CIC-DDoS2019 dataset, these two classes show significant misclassification and low recall, despite the fact that the models achieve high overall accuracy. Focusing specifically on SSDP and UDP classes in the CIC-DDoS2019 dataset reveals very distinct but related causes of their poor performance in machine learning intrusion detection systems. SSDP generates bursty and irregular traffic, causing models trained on stable patterns to fail. Attack traffic for this class may differ drastically between training and testing splits due to time-based collection. The heavy overlap of malicious SSDP traffic with benign UDP traffic in basic flow features presents a significant challenge for network intrusion detection systems (NIDS). Attackers intentionally leverage legitimate network protocols and infrastructure, making malicious traffic look deceptively normal when analyzing basic flow metrics like source/destination ports. SSDP and UDP attacks generate high volumes of UDP traffic with similar flow characteristics (e.g., high Total Fwd Packets, low Avg Packet Size). They

TABLE 7. Benefits of D-optimal random sub-sampling technique.

Feature	Description
Fast	Only evaluates 5,000 samples to generate random binary masks
Memory-safe	No attempt to construct 2^{47} masks for 47 features
Deterministic (with seed)	Reproducible top-k selections
Approximate	Still good for initial island/agent design in Jaya population
Scalable	Scalable to 50+ features

TABLE 8. Comparative scalability vs. Alternatives.

Algorithm	Memory Scaling (w.r.t. features)	Parallel Scalability (cores)	Notes
Standard Jaya	$O(ND)$	Low	Global memory shared
Multi-Island Jaya (MIJ)	$O(ND/m)$	High	Best trade-off

struggle in a fine-grained multiclass scenario and misclassify one specific UDP type as another.

C. MEMORY EFFICIENCY AND SCALABILITY OF THE MULTI-ISLAND JAYA ALGORITHM

Here is the quantitative evidence and analysis supporting the memory efficiency and scalability of the Multi-Island Jaya Algorithm (MI-Jaya) for feature selection.

Initial parameters of D-optimal downsampling are set as follows:

- 1) Number of features $F=47$
- 2) Number of selected topmost masks $k=20$
- 3) Total samples $S=5000$
- 4) Minimum number of features to be selected $=2$
- 5) Seed=42

D-optimal downsampling generates 5000 initial random binary samples with 47 bits corresponding to 47 features in the preprocessed dataset, where the bit value 0 represents the rejection of the corresponding feature and the bit value 1 represents the selection of the corresponding feature. This method can be applied to more than 50 features. The minimum number of features to be selected is restricted to 2. The top 20 masks are Used to select 20 optimal feature subsets from the entire random sample $S=5000$. Random seed s is set to 42 for reproducibility. The benefits of the D-optimal Random Sub-sampling technique are listed in table 7. Each island operates asynchronously, allowing $O(N/m)$ memory load per island where N = population size= 70% of all records in the entire dataset (i.e., records in training data = $259641 \times 0.70 = 1,81,748$) and m = number of islands= 4. The comparison of scalability with other algorithms is tabulated in table 8. The memory requirement of the standard Jaya algorithm with respect to features or dimensions D is of the $O(ND)$ order, while that

of the proposed Multi-Island Jaya (MIJ) is of the $O(ND/m)$ order.

V. CONCLUSION AND FUTURE DIRECTION

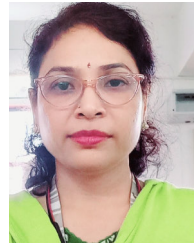
This study integrates the advantages of the D-optimal Sub-sampling technique with the Jaya feature selection algorithm to introduce a pipeline for a memory-efficient and scalable feature selection method. It offers a scalable approach to select features from the volumetric CIC-DDoS2019 attack traffic associated with large-scale DDoS attacks. The dataset containing the selected features is used to create an optimized automated attack detection model utilizing the H2O AutoML framework. This model has surpassed recent comparable attack detection models in precision, recall, f1 score, and accuracy, achieving an average score of 90% in all metrics.

Looking ahead, similar to the KS-DDoS and SSK-DDoS models, the proposed model can also be implemented and deployed in a real-time distributed processing environment for the prompt prediction of various DDoS attack detections. Additionally, the Jaya algorithm can be explored using a range of effective transformation functions.

REFERENCES

- [1] D. H. Sven and S. Nath. (2024). *A Retrospective on Ddos Trends in 2023 and Actionable Strategies for 2024*. [Online]. Available: <https://www.akamai.com/blog/security/a-retrospective-on-ddos-trends-in-2023>
- [2] G. Singh and N. Khare, "A survey of intrusion detection from the perspective of intrusion datasets and machine learning techniques," *Int. J. Comput. Appl.*, vol. 44, no. 7, pp. 659–669, Jul. 2022.
- [3] K. Rambabu and N. Venkatram, "Ensemble classification using traffic flow metrics to predict distributed denial of service scope in the Internet of Things (IoT) networks," *Comput. Electr. Eng.*, vol. 96, Dec. 2021, Art. no. 107444.
- [4] S. Bhatia, S. Behal, and I. Ahmed, "Distributed denial of service attacks and defense mechanisms: Current landscape and future directions," in *Versatile Cybersecurity*, Cham, Switzerland, 2018, pp. 55–97.
- [5] S. Yu, *Distributed Denial of Service Attack and Defense*. Cham, Switzerland: Springer, 2014.
- [6] G. Singh and N. Khare, "GSFI: A hybrid multiclass classifier for minority attack detection in Internet of Things network," *Int. J. Ad Hoc Ubiquitous Comput.*, vol. 38, no. 1, pp. 45–61, 2021.
- [7] N. Khare, P. Devan, C. Chowdhary, S. Bhattacharya, G. Singh, S. Singh, and B. Yoon, "SMO-DNN: Spider monkey optimization and deep neural network hybrid classifier model for intrusion detection," *Electronics*, vol. 9, no. 4, p. 692, Apr. 2020.
- [8] N. Khare and P. Viswanathan, "Decision tree-based fraud detection mechanism by analyzing uncertain data in banking system," in *Emerging Research in Data Engineering Systems and Computer Communications: Proceedings of CCODE 2019*. Cham, Switzerland: Springer, 2020, pp. 79–90.
- [9] R. Gopi, V. Sathiyamoorthi, S. Selvakumar, R. Manikandan, P. Chatterjee, N. Z. Jhanjhi, and A. K. Luhach, "Enhanced method of ANN based model for detection of DDoS attacks on multimedia Internet of Things," *Multimedia Tools Appl.*, vol. 81, no. 19, pp. 26739–26757, Aug. 2022.
- [10] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Secur. Technol. (ICCST)*, Oct. 2019, pp. 1–8.
- [11] R. Venkata Rao, "Jaya: A simple and new optimization algorithm for solving constrained and unconstrained optimization problems," *Int. J. Ind. Eng. Comput.*, vol. 7, no. 1, pp. 19–34, 2016.
- [12] R. A. Zitar, M. A. Al-Betar, M. A. Awadallah, I. A. Doush, and K. Assaleh, "An intensive and comprehensive overview of JAYA algorithm, its versions and applications," *Arch. Comput. Methods Eng.*, vol. 29, no. 2, pp. 763–792, Mar. 2022.

- [13] V. V. Fedorov, "Optimal design with bounded density: Optimization algorithms of the exchange type," *J. Stat. Planning Inference*, vol. 22, no. 1, pp. 1–13, May 1989.
- [14] V. V. Fedorov, *Theory of Optimal Experiments*. Amsterdam, The Netherlands: Elsevier, 2013.
- [15] A. Atkinson, A. Donev, and R. Tobias, *Optimum Experimental Designs, With SAS*. London, U.K.: Oxford Univ. Press, 2007.
- [16] T. Reuter and R. Schwabe, "Optimal subsampling design for polynomial regression in one covariate," *Stat. Papers*, vol. 64, no. 4, pp. 1095–1117, Aug. 2023.
- [17] E. LeDell and S. Poirier, "H2O AutoML: Scalable automatic machine learning," in *Proc. AutoML Workshop ICML*, 2020, p. 24.
- [18] A. Ahmim, F. Maazouzi, M. Ahmim, S. Namane, and I. B. Dhaou, "Distributed denial of service attack detection for the Internet of Things using hybrid deep learning model," *IEEE Access*, vol. 11, pp. 119862–119875, 2023.
- [19] J. Zhao, Y. Liu, Q. Zhang, and X. Zheng, "CNN-AttBiLSTM mechanism: A DDoS attack detection method based on attention mechanism and CNN-BiLSTM," *IEEE Access*, vol. 11, pp. 136308–136317, 2023.
- [20] O. Polat, A. Ayid Ahmad, S. Oyucu, E. Algül, F. Doğan, and A. Aksöz, "Temporal-spatial feature extraction in IoT-based SCADA system security: Hybrid CNN-LSTM and attention-based architectures for malware classification and attack detection," *IEEE Access*, vol. 13, pp. 102109–102132, 2025.
- [21] K. Gogia, R. Narula, K. Bansal, and A. Arora, "Comparative study of feature selection methods for deep learning-based intrusion detection in IoT networks," in *Proc. 6th Int. Conf. Intell. Commun. Technol. Virtual Mobile Netw. (ICICV)*, 2025, pp. 1733–1738.
- [22] F. O. Albasheer, R. R. Haibatti, M. Agarwal, and S. Y. Nam, "A novel IDS based on Jaya optimizer and smote-ENN for cyberattacks detection," *IEEE Access*, vol. 12, pp. 101506–101527, 2024.
- [23] U. N. Dulhare, A. Mubeen, and K. Ahmad, "Hands-on H2O machine learning tool," in *Machine Learning and Big Data: Concepts, Algorithms, Tools and Applications*. Hoboken, NJ, USA, 2020, pp. 423–453.
- [24] D. Cook, *Practical Machine Learning With H2O: Powerful, Scalable Techniques for Deep Learning and AI*. Sebastopol, CA, USA: O'Reilly Media, 2016.
- [25] S. Aiello, C. Click, H. Roark, L. Rehak, and P. Stetsenko, "Machine learning with Python and H2O," H2O.Ai, Inc., Mountain View, CA, USA, Tech. Rep., 2016.
- [26] N. S. Alotaibi, H. I. Sayed Ahmed, S. O. M. Kamel, and G. F. ElKabbany, "Secure enhancement for MQTT protocol using distributed machine learning framework," *Sensors*, vol. 24, no. 5, p. 1638, Mar. 2024.
- [27] I. Vaccari, G. Chiola, M. Aiello, M. Mongelli, and E. Cambiaso, "MQTTset, a new dataset for machine learning techniques on MQTT," *Sensors*, vol. 20, no. 22, p. 6578, Nov. 2020.
- [28] N. V. Patil, C. R. Krishna, and K. Kumar, "KS-DDoS: Kafka streams-based classification approach for DDoS attacks," *J. Supercomput.*, vol. 78, no. 6, pp. 8946–8976, Apr. 2022.
- [29] N. V. Patil, C. R. Krishna, and K. Kumar, "SSK-DDoS: Distributed stream processing framework based classification system for DDoS attacks," *Cluster Comput.*, vol. 25, no. 2, pp. 1355–1372, Apr. 2022.
- [30] A. Singh, J. Amutha, J. Nagar, S. Sharma, and C. Lee, "AutoML-ID: Automated machine learning model for intrusion detection using wireless sensor network," *Sci. Rep.*, vol. 12, no. 1, p. 9074, 2022.
- [31] W. Elmasry, A. Akbulut, and A. H. Zaim, "Evolving deep learning architectures for network intrusion detection using a double PSO Metaheuristic," *Comput. Netw.*, vol. 168, Feb. 2020, Art. no. 107042.
- [32] M. A. Khan, N. Iqbal, Imran, H. Jamil, and D.-H. Kim, "An optimized ensemble prediction model using AutoML based on soft voting classifier for network intrusion detection," *J. Netw. Comput. Appl.*, vol. 212, Mar. 2023, Art. no. 103560.
- [33] S. Szabó, I. J. Holb, V. É. Abriha-Molnár, G. Szatmári, S. K. Singh, and D. Abriha, "Classification assessment tool: A program to measure the uncertainty of classification models in terms of class-level metrics," *Appl. Soft Comput.*, vol. 155, Apr. 2024, Art. no. 111468.



GEETA SINGH received the master's degree in computer science from Barkatullah University, Bhopal, in 2006, and the master's degree in software systems from RGPV, Bhopal, in 2014. She was an Assistant Professor with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India, for three years. Overall, she has more than ten years of academic and research experience. She is currently a Research Scholar with the School

of Computer Science and Engineering, Vellore Institute of Technology. She has published many research articles in national and international journals. Her research interests include information security and machine learning techniques.



NEELU KHARE received the Ph.D. degree from the Maulana Azad National Institute of Technology Bhopal, India, in 2011. She has been a Professor with the School of Computer Science Engineering and Information Systems, VIT, Vellore, India, since 2012. Overall, she has 16 years of academic and research experience. She has published 55 research papers in international journals and conferences and six book chapters. She has guided six Ph.D. students. She published

two patents and completed the research project funded by King Saud University, Saudi Arabia, worth 20 lacs. Her research interests include data science, machine learning, deep learning, soft computing techniques, network intrusion detection, the IoT, and bio-informatics.

...